

CSC 480: Artificial Intelligence

3.2 - Tree Search - Informed Search

Rodrigo Canaan
Assistant Professor
Computer Science Department
Cal Poly, San Luis Obispo
rcanaan@calpoly.edu

Jan 30, 2026

Informed Search

Informed Search

- ❖ **relies on additional knowledge about the problem or domain**
 - ❖ frequently expressed through heuristics (“rules of thumb”)
- ❖ **used to distinguish more promising paths towards a goal**
 - ❖ may be mislead, depending on the quality of the heuristic
- ❖ **in general, performs much better than uninformed search**
 - ❖ but frequently still exponential in time and space for realistic problems

Informed Search

In this lecture, we will discuss:

- ❖ A general approach for informed search called **best-first search**
- ❖ A few instantiations of this approach:
 - ❖ Greedy best-first search
 - ❖ A^*
 - ❖ Some memory-bounded variations of Greedy and A^*
 - ❖ Also MCTS
- ❖ Ways to generate heuristics for informed search

Best-First Search

- ❖ **relies on an evaluation function $f(n)$ that gives an indication of how useful it would be to expand a node**
 - ❖ family of search methods with various evaluation functions
 - ❖ often based on heuristics, but can use other information as well (see: A*)
 - ❖ the evaluation function is different from the cost function $g(n)$ in Uniform-Cost Search
 - ❖ cost function: path cost calculated from known costs for steps along the path to the current node
 - ❖ evaluation function: estimate of the cost to the goal
- ❖ **the node with the lowest value is expanded first**
 - ❖ the “best” in the name is a little misleading: the node with the lowest value for the evaluation function is not necessarily one that is on an optimal path to a goal
 - ❖ if we really know which one is the best, there’s no need to do a search

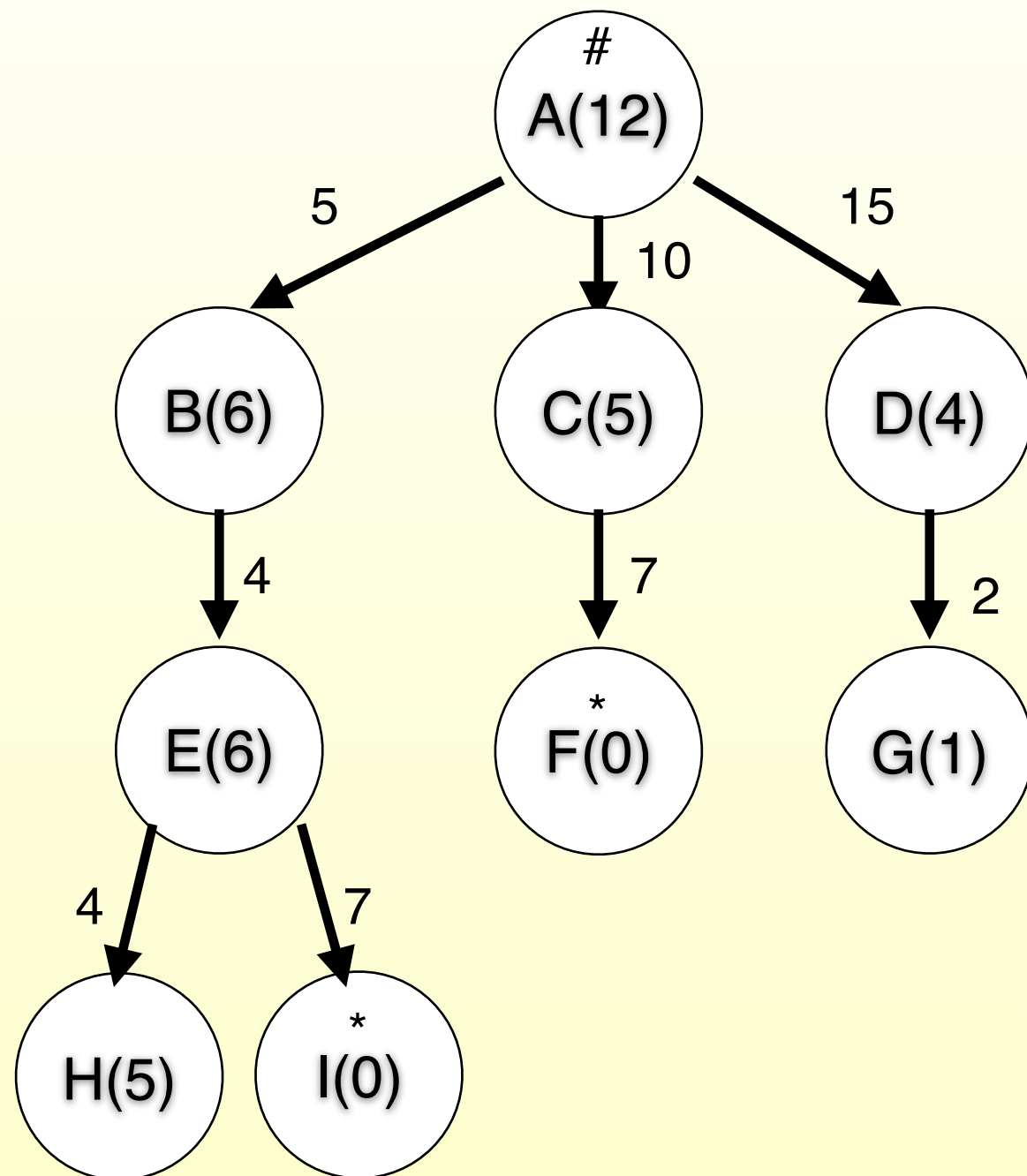
```
function BEST-FIRST-SEARCH(problem, EVAL-FN) returns solution  
    fringe := queue with nodes ordered by EVAL-FN  
    return TREE-SEARCH(problem, fringe)
```

Greedy Best-First Search

- ❖ **minimizes the estimated cost to a goal**
 - ❖ expand the node that seems to be closest to a goal
 - ❖ utilizes a heuristic function as evaluation function
 - ❖ $f(n) = h(n)$ = estimated cost from the current node to a goal
 - ❖ heuristic functions are problem-specific
 - ❖ often straight-line distance for route-finding and similar problems
- ❖ often better than depth-first, although worst-cases complexities are equal or worse (space)

```
function GREEDY-SEARCH(problem) returns solution  
    return BEST-FIRST-SEARCH(problem, h)
```

Example State Space



Legend:

- Starting state

* - Goal state

● Visited (expanded)

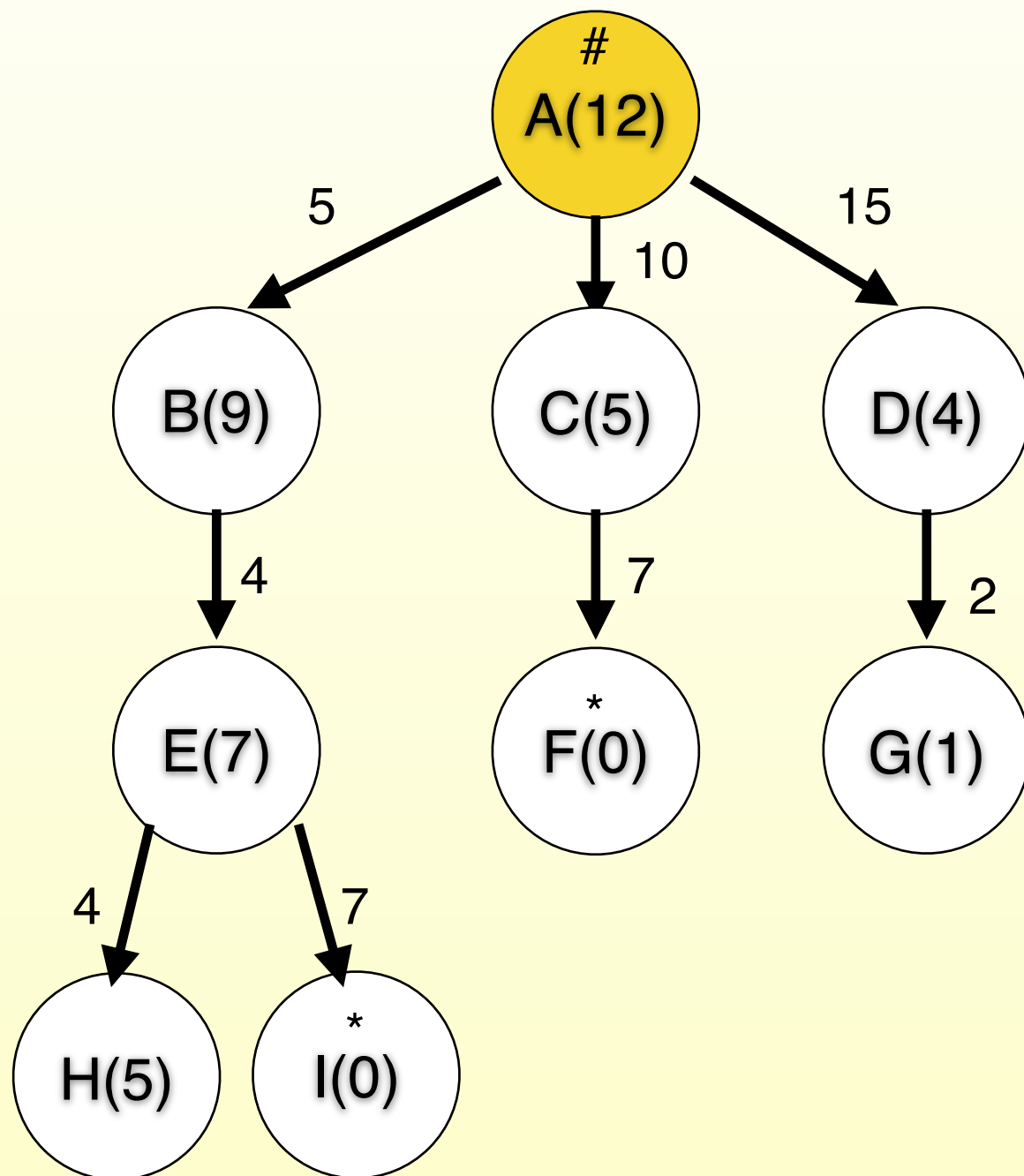
● Current state

● Fringe

$\xrightarrow{c}$ Transition cost = c

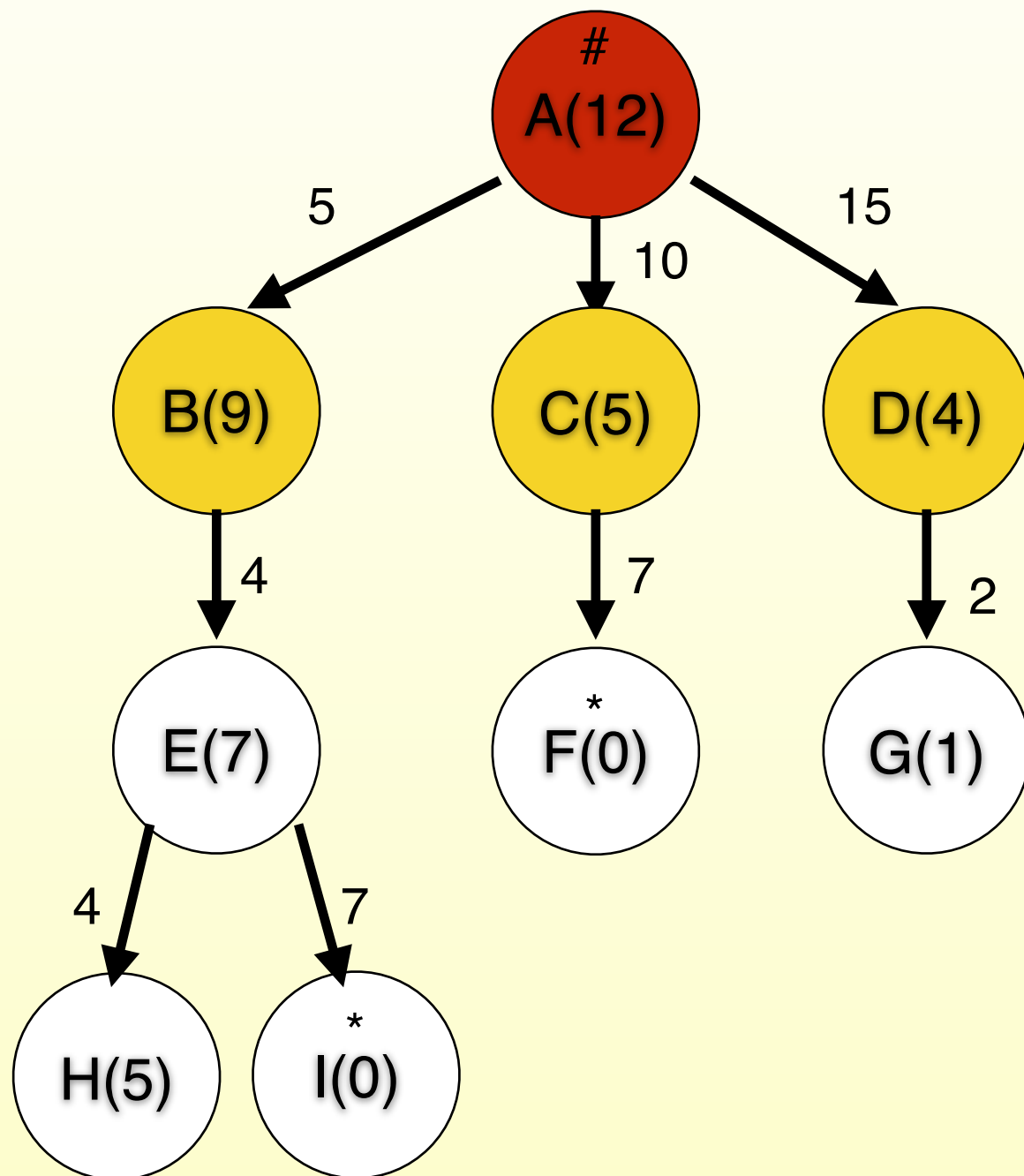
(h) Heuristic = h

Greedy Best-First Example



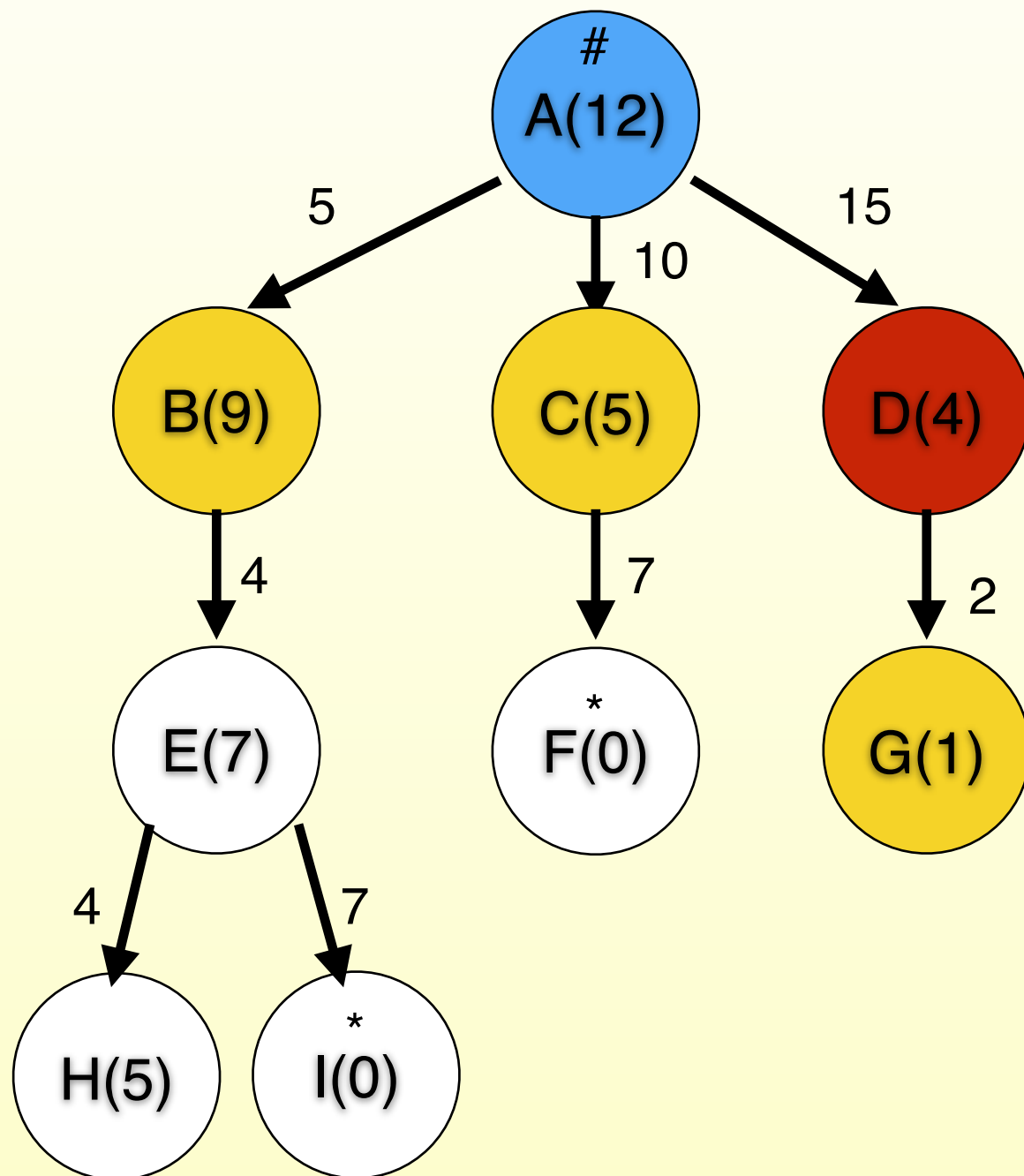
Node	f = h
A	12

Greedy Best-First Example



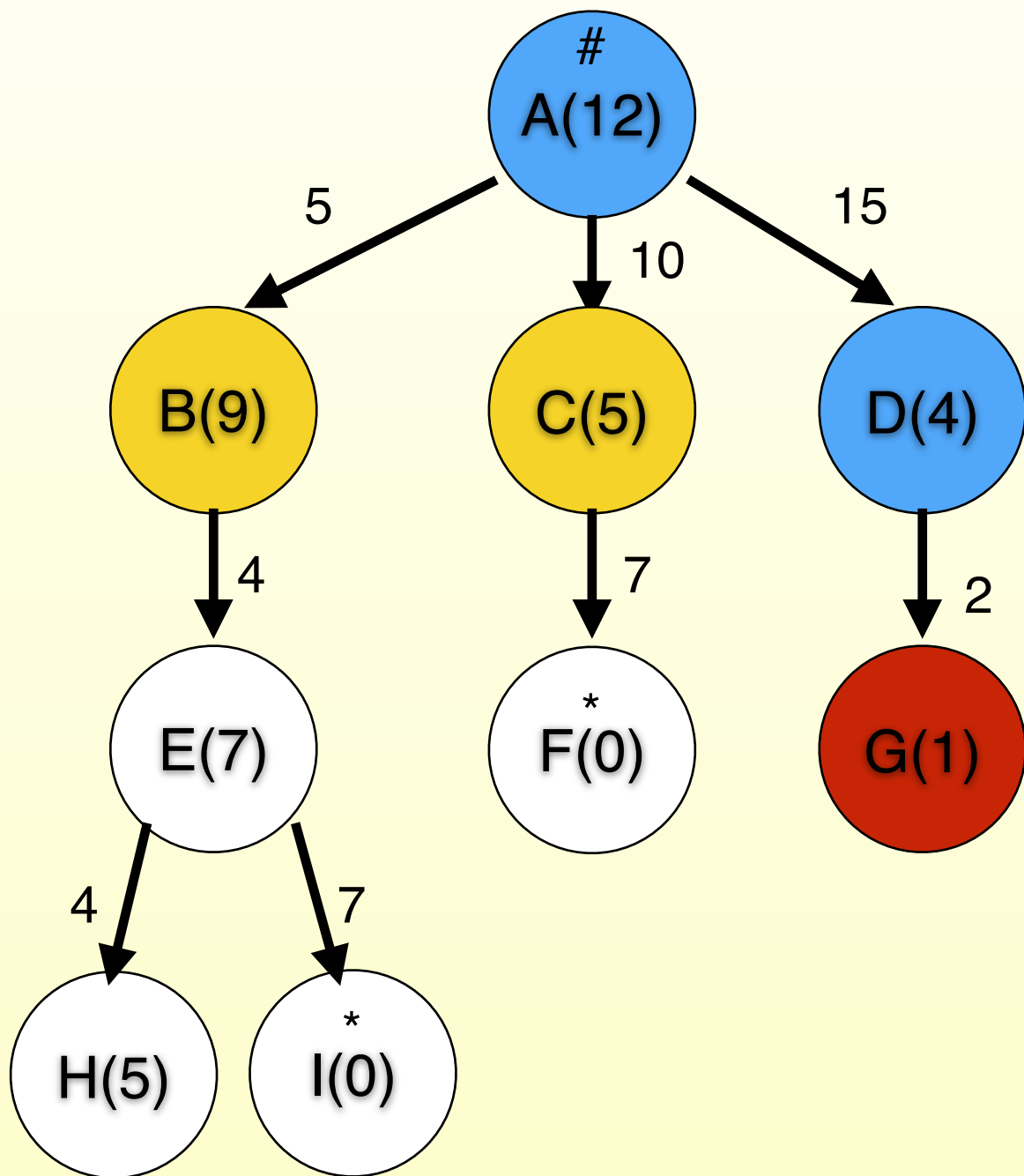
Node	f = h
D	4
C	5
B	9

Greedy Best-First Example



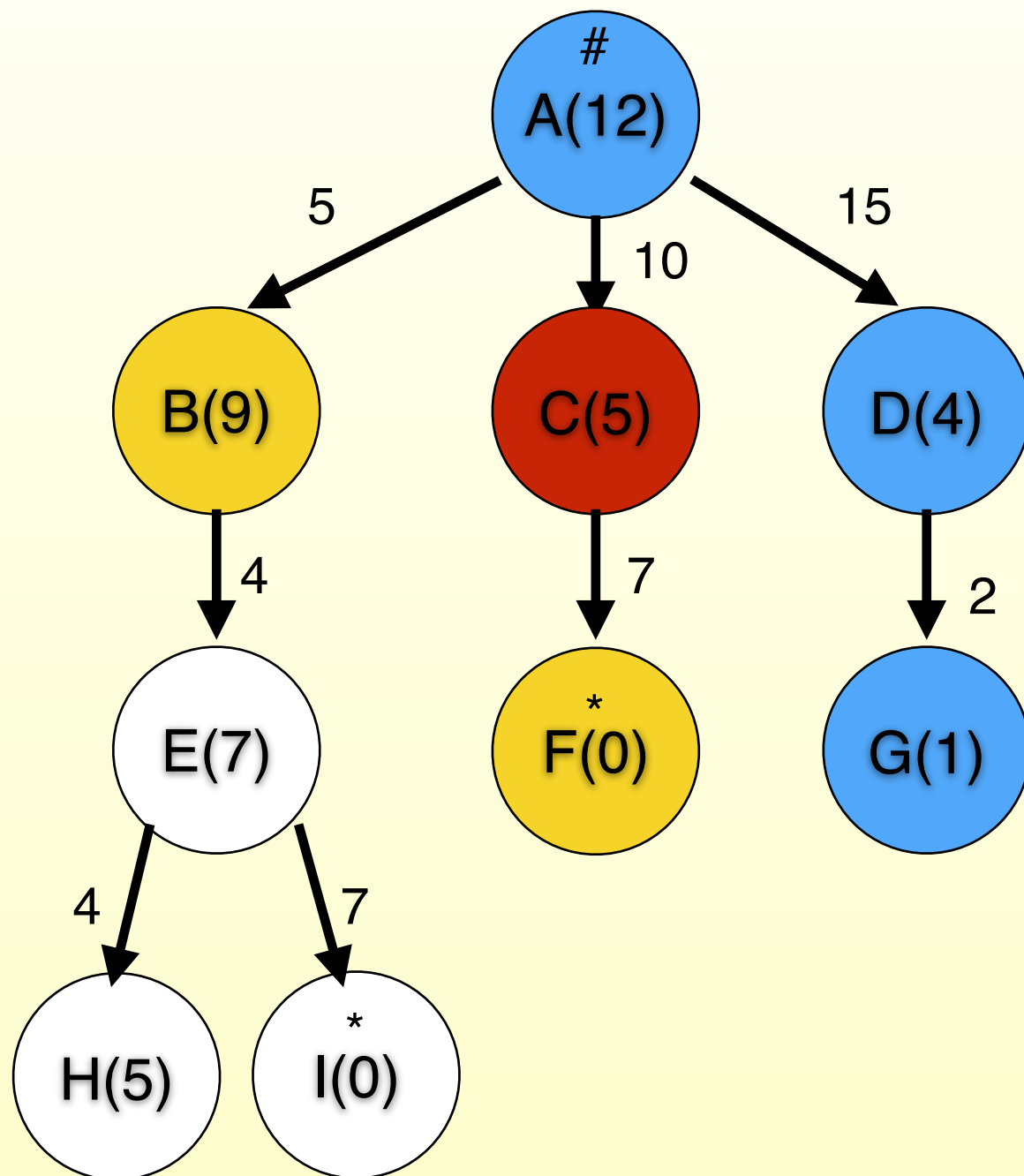
Node	f = h
G	1
C	5
B	9

Greedy Best-First Example



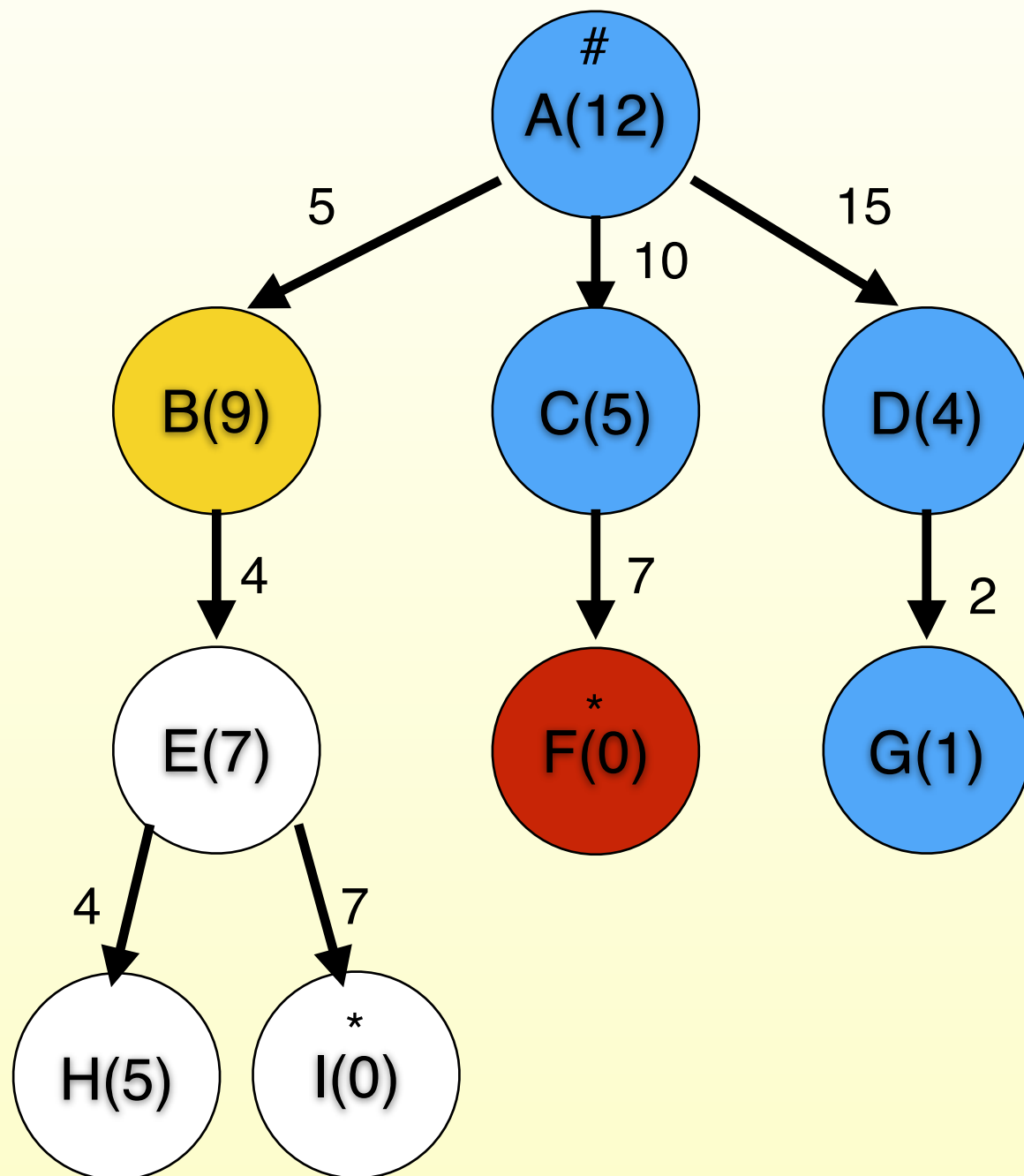
Node	f = h
C	5
B	9

Greedy Best-First Example



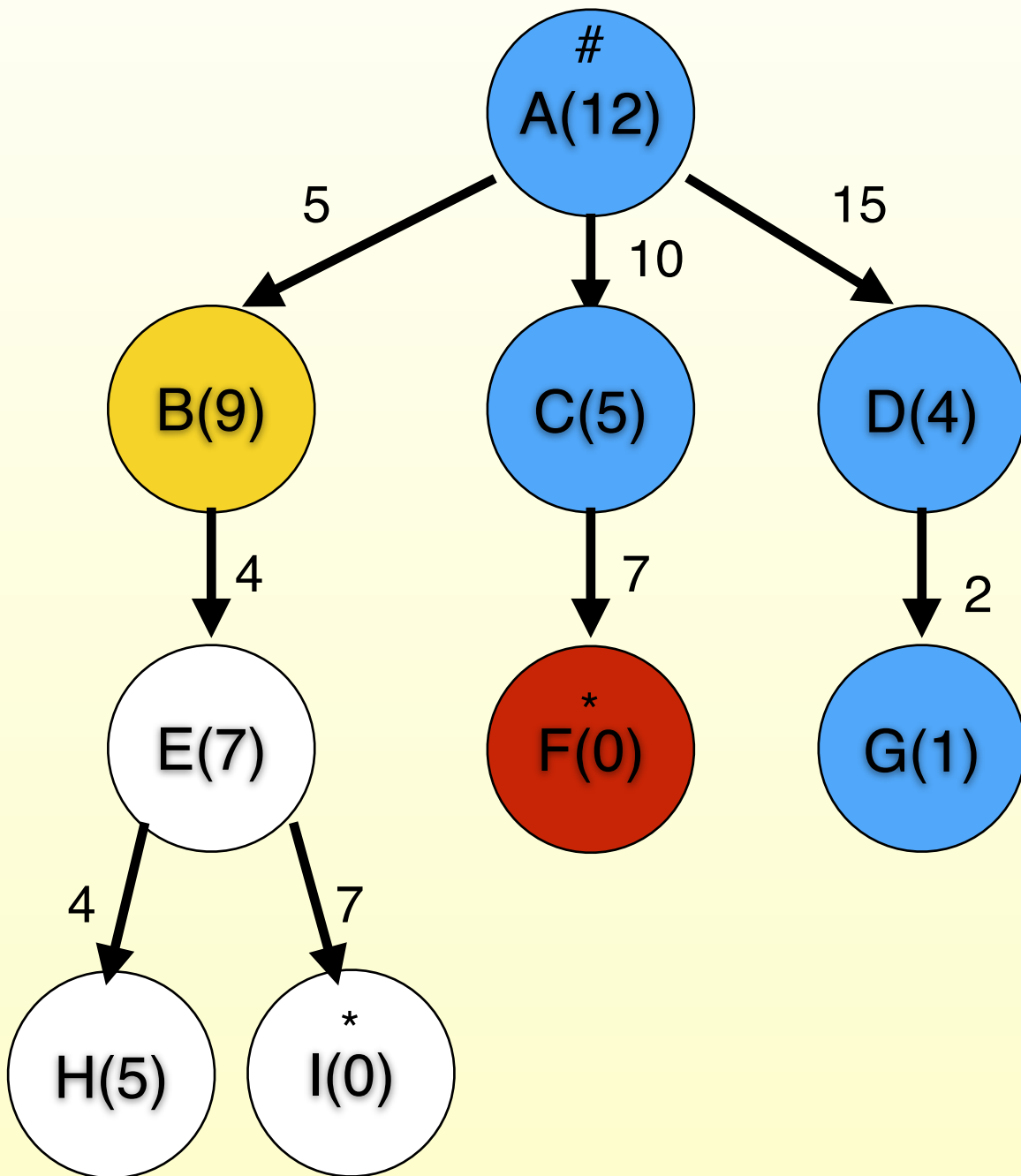
Node	$f = h$
F	0
B	9

Greedy Best-First Example



Node	f = h
B	9

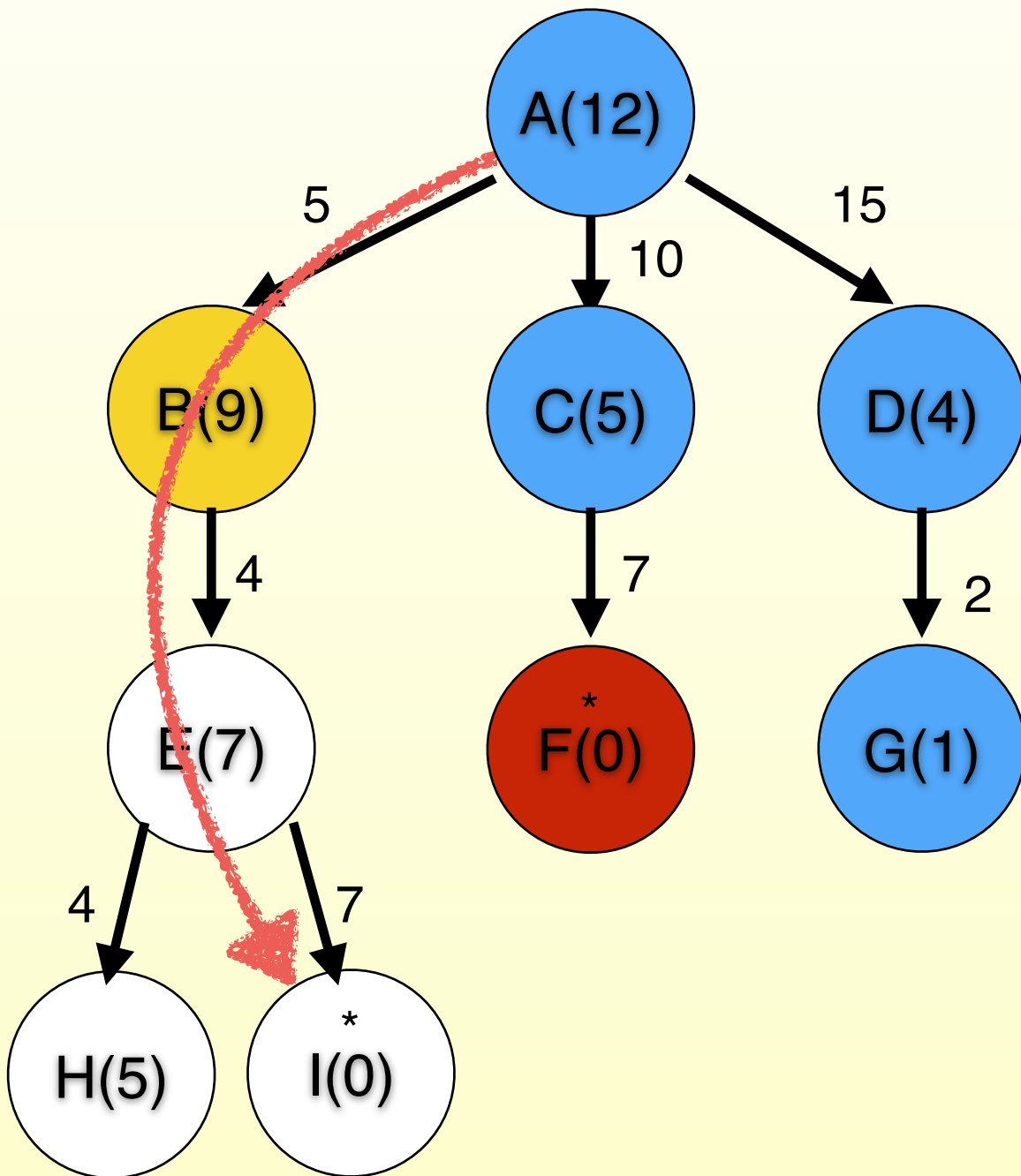
Greedy Best-First Example



Node	$f = h$
B	9

Goal is found agent returns path ACF with cost 17

Greedy is not Optimal!

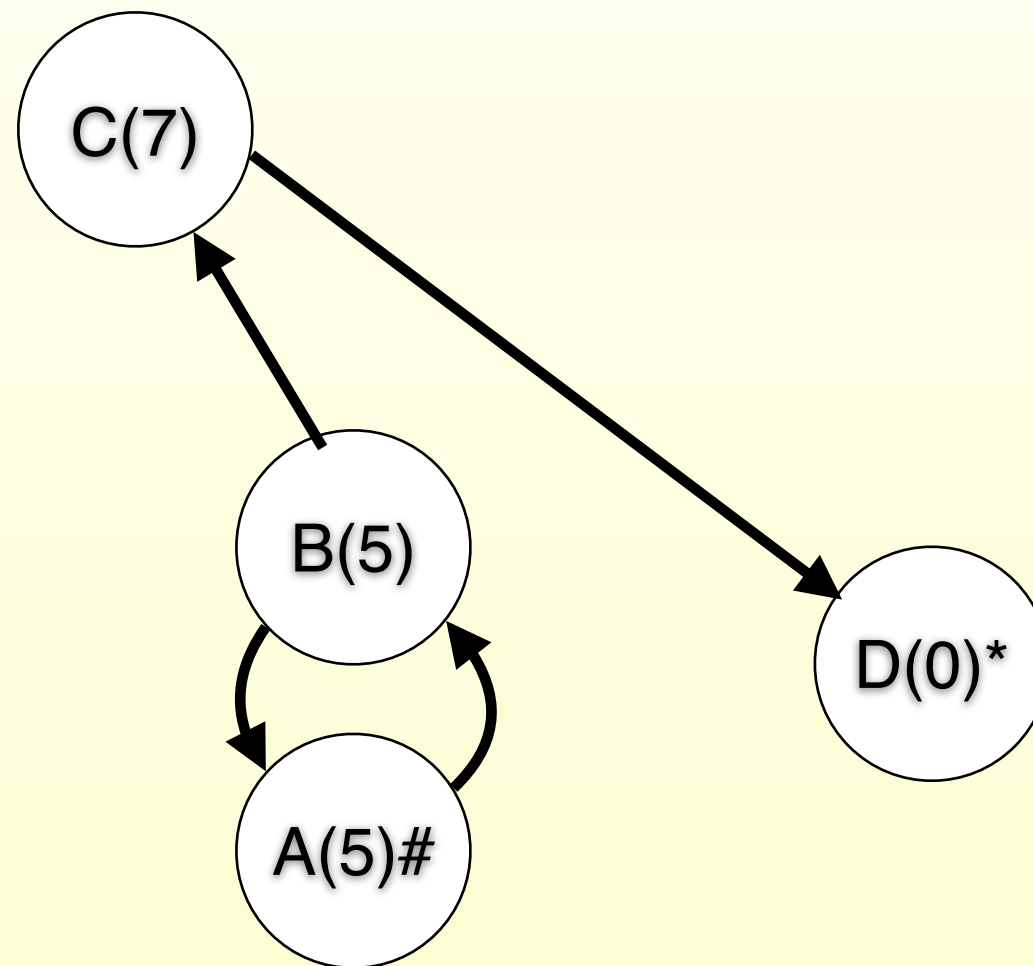


Node	$f = h$
B	9

A cheaper path, ABEI exists with cost 16!

A cheaper path, ABEI exists with cost 16!

Greedy is also incomplete!



Has the same problems with loops and infinite paths as Depth-First Search

Greedy Best-First Search

- ❖ **minimizes the estimated cost to a goal**
 - ❖ expand the node that seems to be closest to a goal
 - ❖ utilizes a heuristic function as evaluation function
 - ❖ $f(n) = h(n)$ = estimated cost from the current node to a goal
 - ❖ heuristic functions are problem-specific
 - ❖ often straight-line distance for route-finding and similar problems
- ❖ often better than depth-first, although worst-cases complexities are equal or worse (space)

```
function GREEDY-SEARCH(problem) returns solution  
  return BEST-FIRST-SEARCH(problem, h)
```

Completeness	Time Complexity	Space Complexity	Optimality
no	b^m	b^m	no

b: branching factor, d: depth of the solution, m: maximum depth of the search tree, l: depth limit

Why use Greedy Best-First Search?

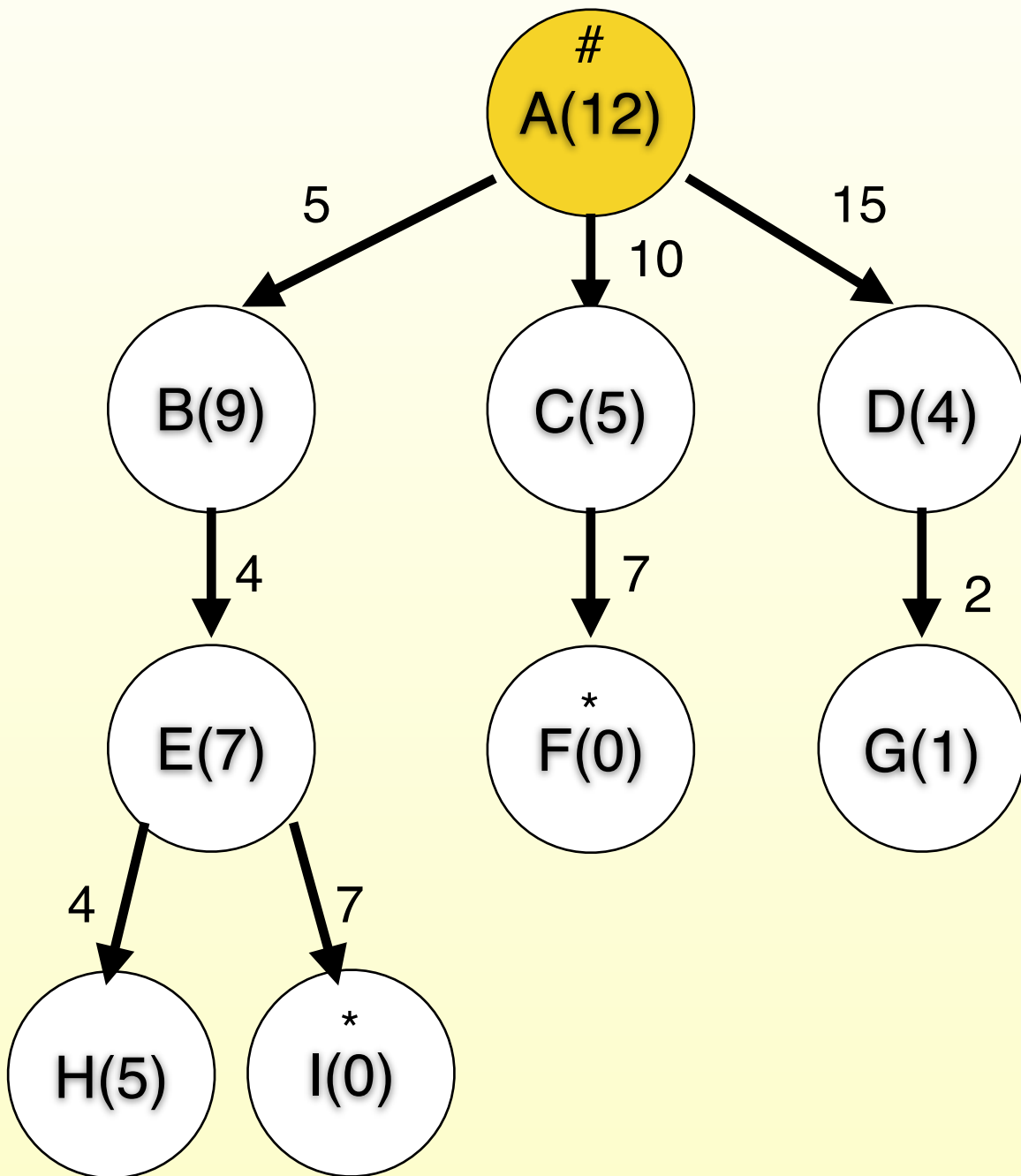
- ❖ If heuristic is known to be very good
- ❖ Faster than A^* (if a solution can be found in the same approximate number of steps)
- ❖ Less memory than A^* (ditto)
- ❖ If cost of search is more important than optimality of solution (cost to current node is essentially sunk cost)

A* Search

- ❖ combines greedy and uniform-cost search to find the (estimated) cheapest path through a prospective node to expand
 - ❖ $f(n) = g(n) + h(n)$
= path cost + estimated cost to the goal
 - ❖ very good search method, but still with (worst-case) complexity problems

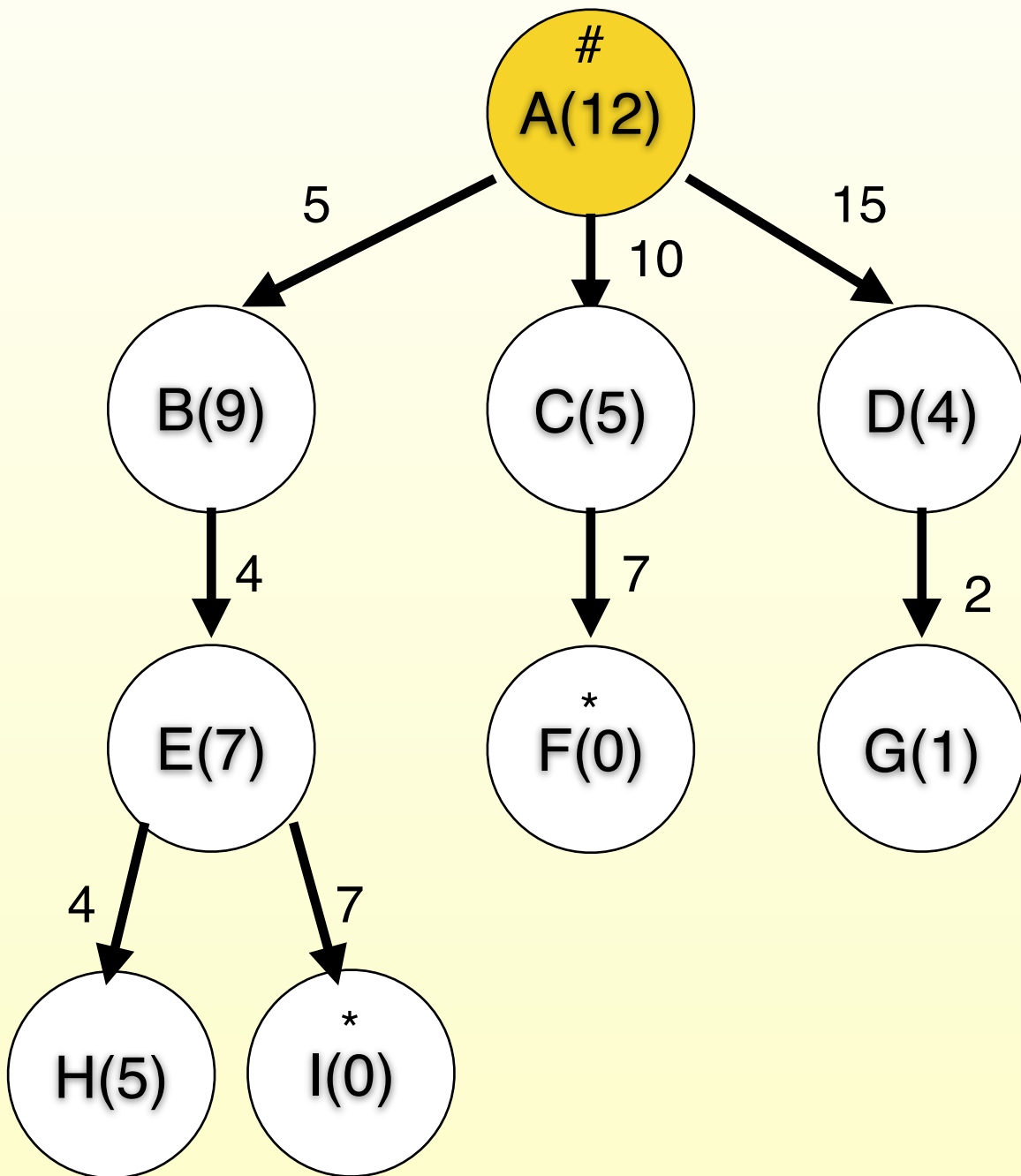
```
function A*-SEARCH(problem) returns solution  
  
    return BEST-FIRST-SEARCH(problem, g+h)
```

A* search example



A* search example

g = How much did it actually cost me to get here?



f = How much do I think it will cost me to reach the goal via this node?

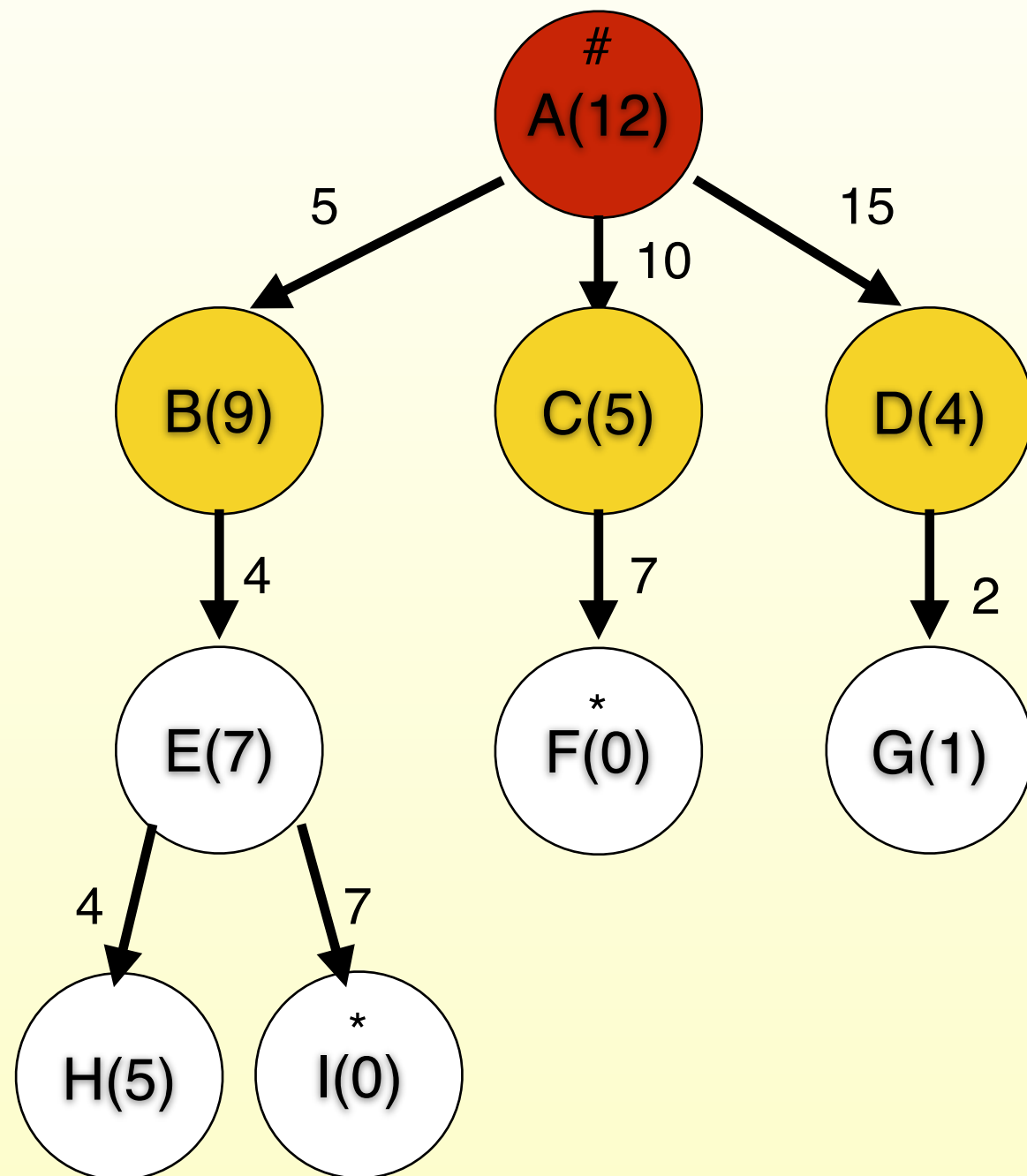
h = How far do I think I am from the goal?

Node	f	g	h
A	12	0	12

How much do I think cost me to reach goal via this node?

h = How far do I think I am from the goal?

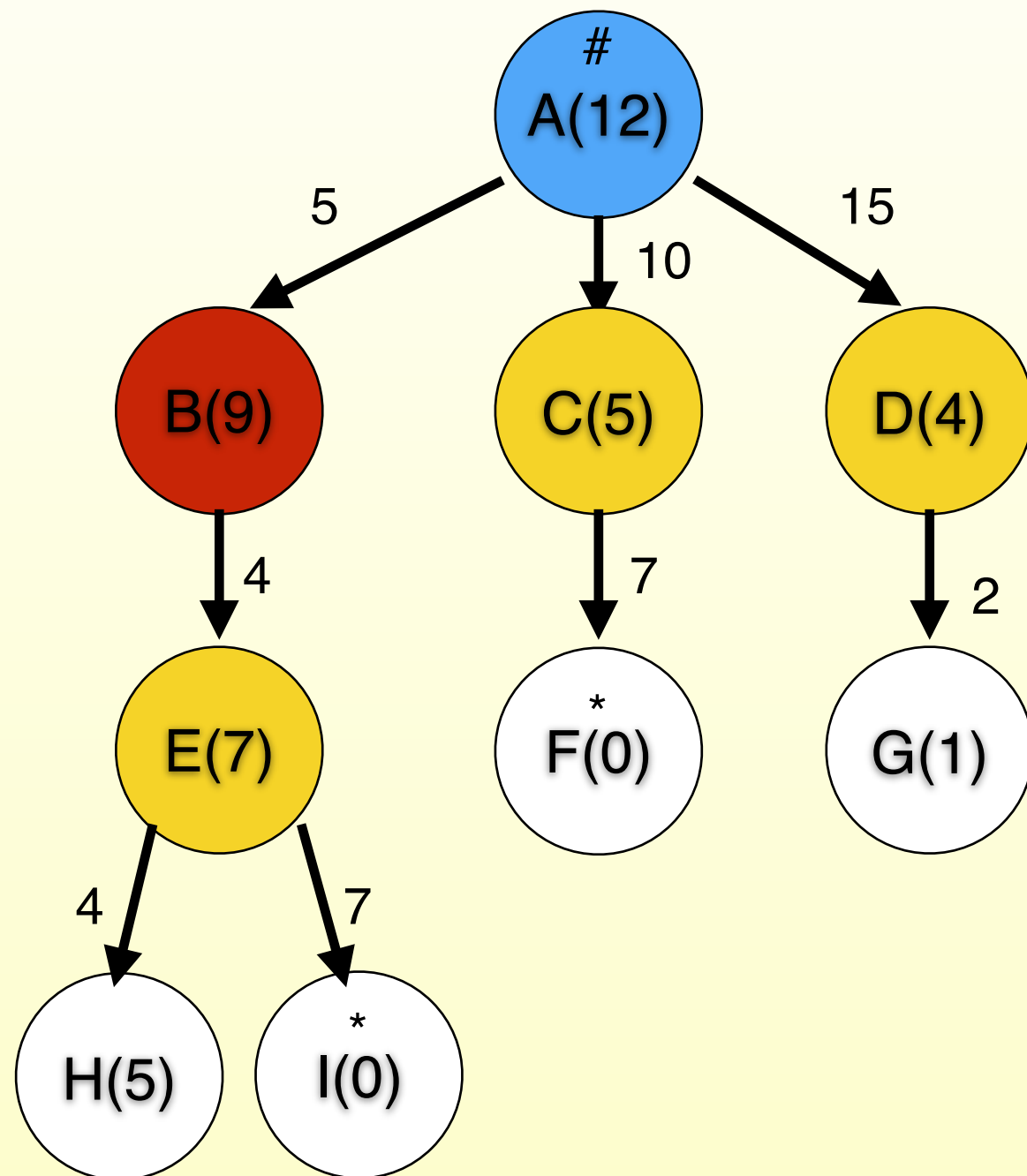
A* search example



Current node: A (g=0)

Node	f	g	h
B	14	5	9
C	15	10	5
D	19	15	4

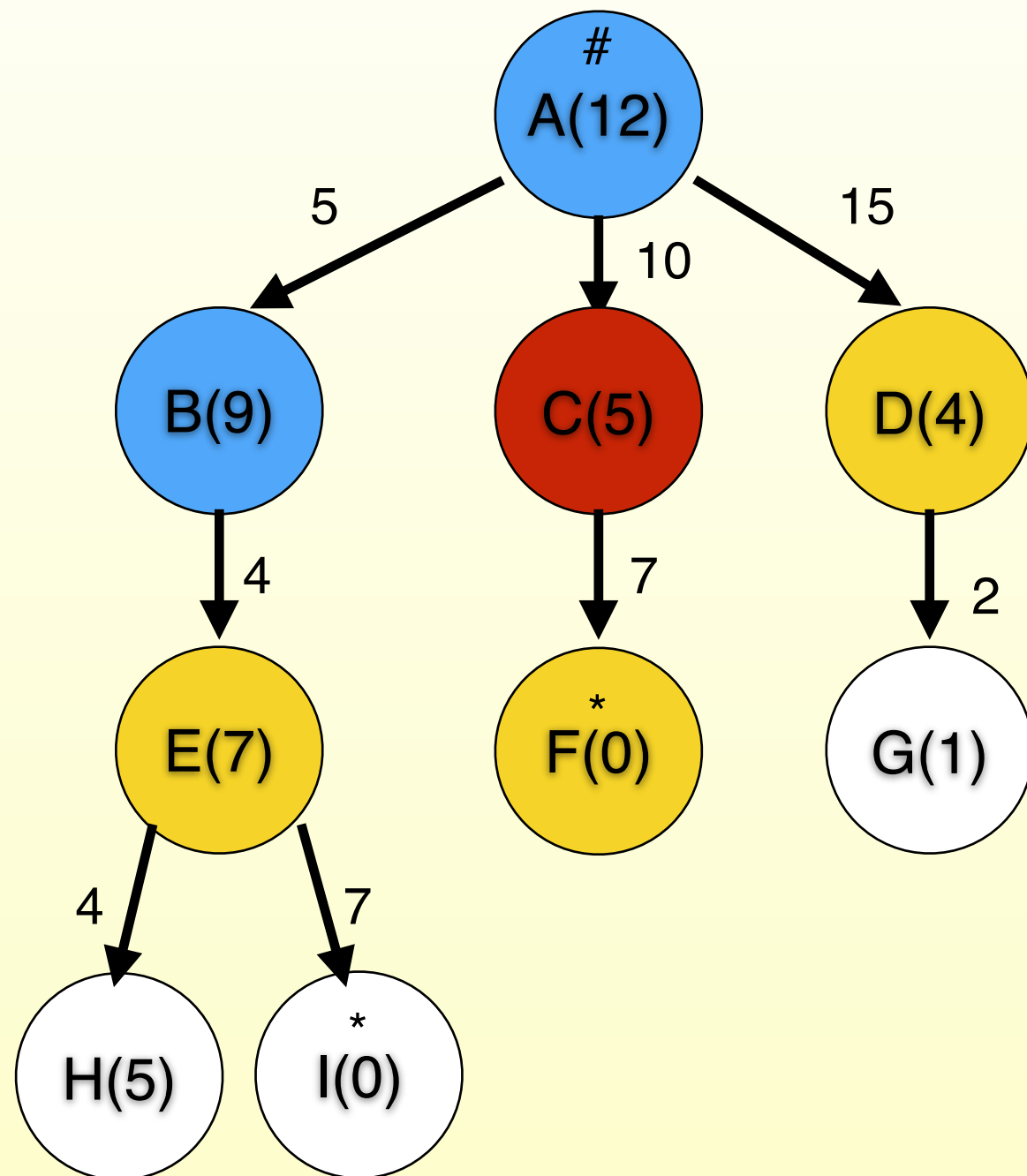
A* search example



Current node: B (g=5)

Node	f	g	h
C	15	10	5
E	16	5+4=9	7
D	19	15	4

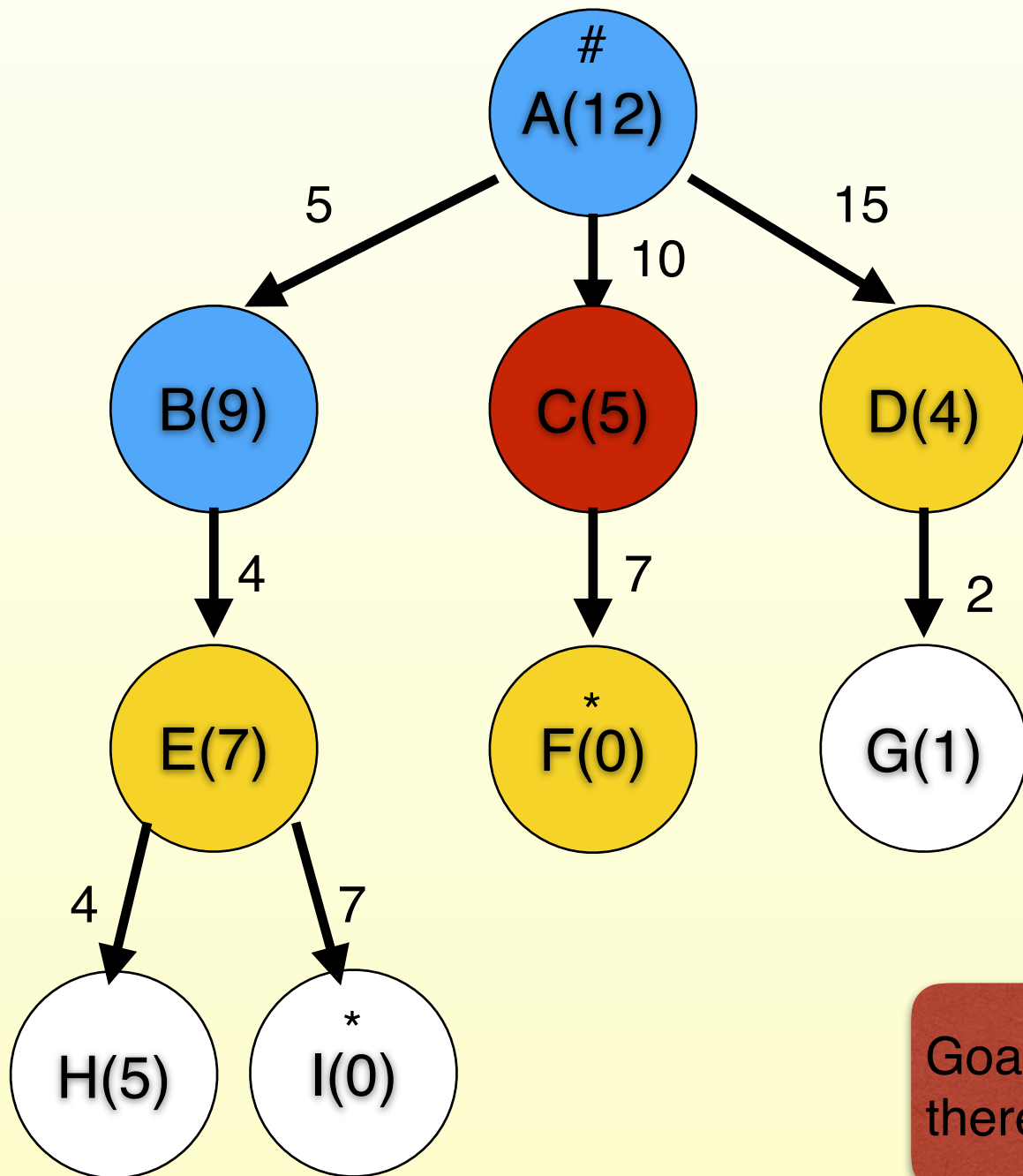
A* search example



Current node: C (g=10)

Node	f	g	h
E	16	9	7
F	17	10+7=17	0
D	19	15	4

A* search example



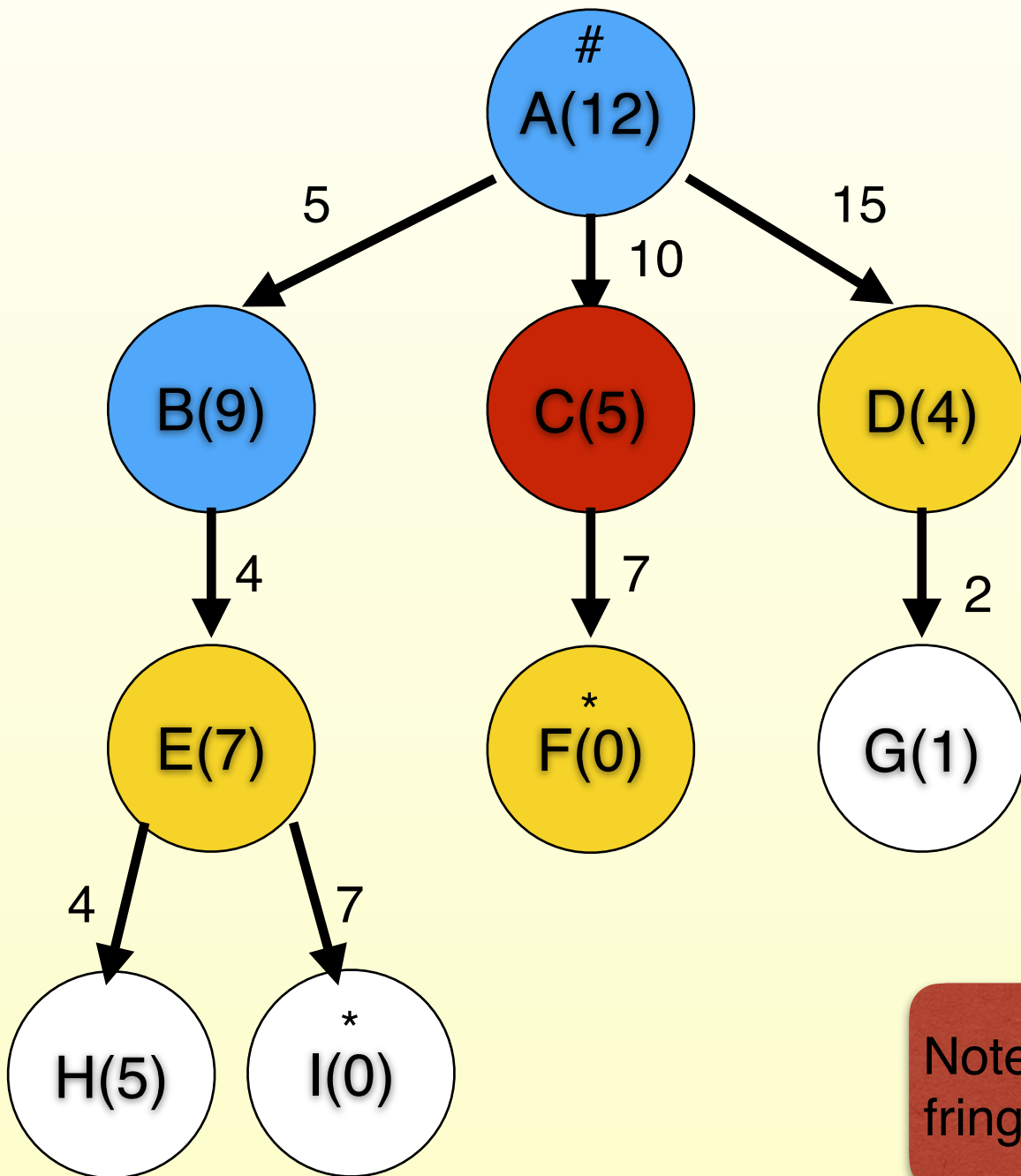
Current node: C (g=10)

Node	f	g	h
E	16	9	7
F	17	10+7=17	0
D	19	15	4

is visible but we cannot explore it yet because
still be a better solution through E

Goal state F is visible but we cannot explore it yet because there might still be a better solution through E

A* search example



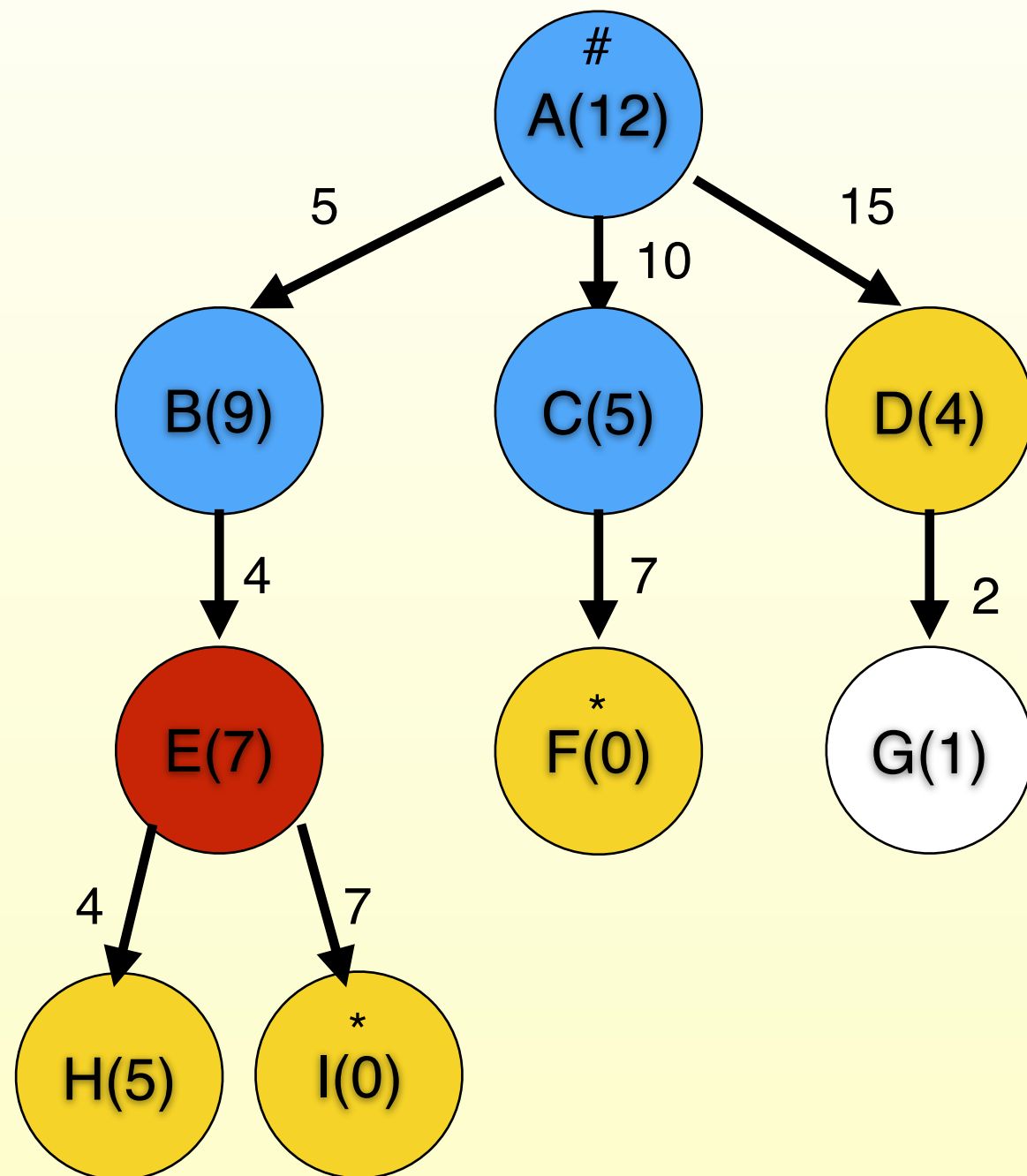
Current node: C (g=10)

Node	f	g	h
E	16	9	7
F	17	10+7=17	0
D	19	15	4

At this point we could drop (prune) D from the

Note that at this point we could drop (prune) D from the fringe

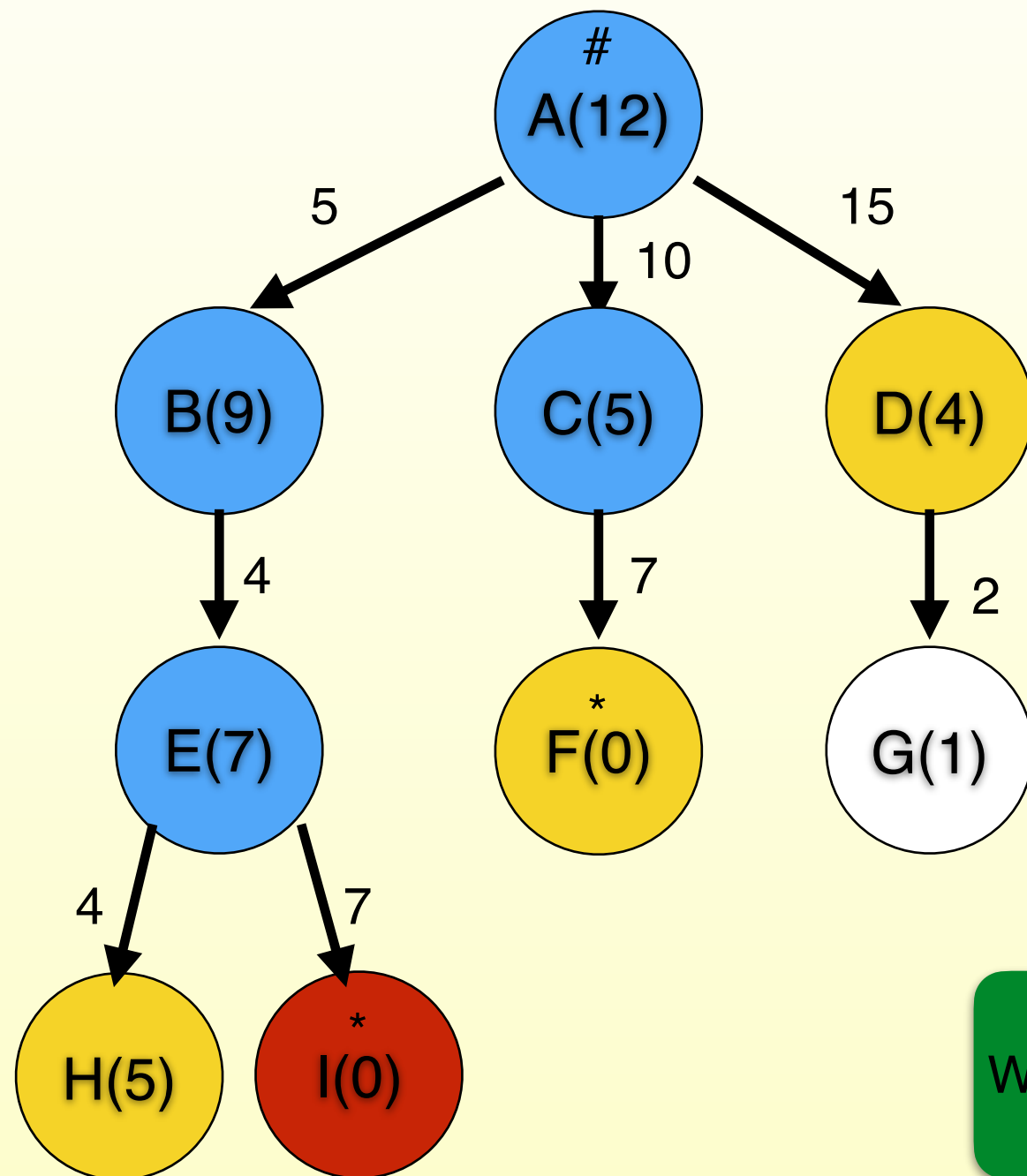
A* search example



Current node: E (g=9)

Node	f	g	h
I	16	9+7=16	0
F	17	10+7=17	0
H	18	9+4=13	5
D	19	15	4

A* search example



Current node: E (g=9)

Node	f	g	h
I	16	9+7=16	0
F	17	10+7=17	0
H	18	13	5
D	19	15	4

We find the goal state I, and return the optimal path ABEI

MARIO: 1024

COINS
11

DIFFICULTY
15

TIME
148

FPS: 24

Attempt: 1 of 1

AStarAgent

Selected Actions:

LEFT

SPEED



A* Search

- ❖ combines greedy and uniform-cost search to find the (estimated) cheapest path through the current node
 - ❖ $f(n) = g(n) + h(n)$
= path cost + estimated cost to the goal
 - ❖ $g(n)$ and $h(n)$ may have to be balanced (normalized)
 - ❖ heuristics must be admissible to guarantee optimality (see next slides)
 - ❖ very good search method, but still with (worst-case) complexity problems

```
function A*-SEARCH(problem) returns solution  
  
    return BEST-FIRST-SEARCH(problem,  $g+h$ )
```

Completeness	Time Complexity	Space Complexity	Optimality
yes	b^d	b^d	yes

b: branching factor, d: depth of the solution, m: maximum depth of the search tree, l: depth limit

Admissible Heuristic

- ❖ **A heuristic is admissible if it never overestimates the cost to the goal.**
 - ❖ In other words, it is “optimistic”
- ❖ **Examples of admissible heuristics**
 - ❖ Euclidean distance, in most maze navigation problems
 - ❖ Number of misplaced tiles in an 8-puzzle (and many puzzles where you move 1 piece at a time)
 - ❖ Also called Hamming Distance

Optimality of A*

With an admissible heuristic:

- ❖ **A* will find the optimal solution**
 - ❖ the first solution found is the optimal one
- ❖ **A* is optimally efficient**
 - ❖ no other optimal algorithm is guaranteed to expand fewer nodes than A*
- ❖ **A* is not always “the best” algorithm**
 - ❖ optimality refers to the expansion of nodes
 - ❖ other criteria might be more relevant
 - ❖ described in abstract Big-O terminology
 - ❖ it generates and keeps all nodes in memory
 - ❖ improved in variations of A*

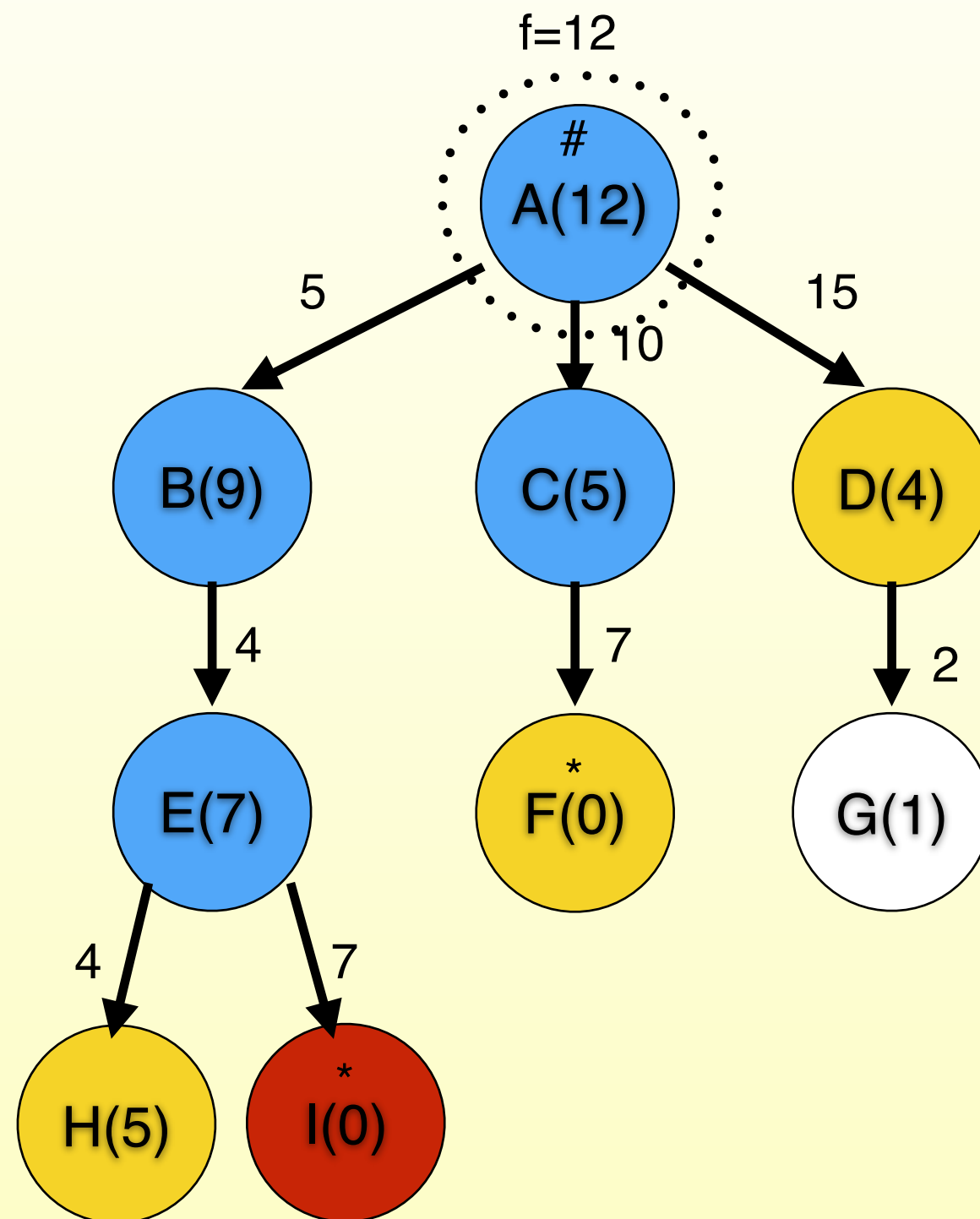
Consistence

- ❖ **A related (stronger) property than admissibility is consistency:**
 - ❖ For all children c of n , $F(n) \leq F(c) + \text{edge}(n,c)$
 - ❖ A form of triangle inequality
- ❖ **With a consistent heuristic, the first time we reach any node is always through the optimal path**
 - ❖ This lets us expand and store even fewer states (since we don't have to worry about duplicates)

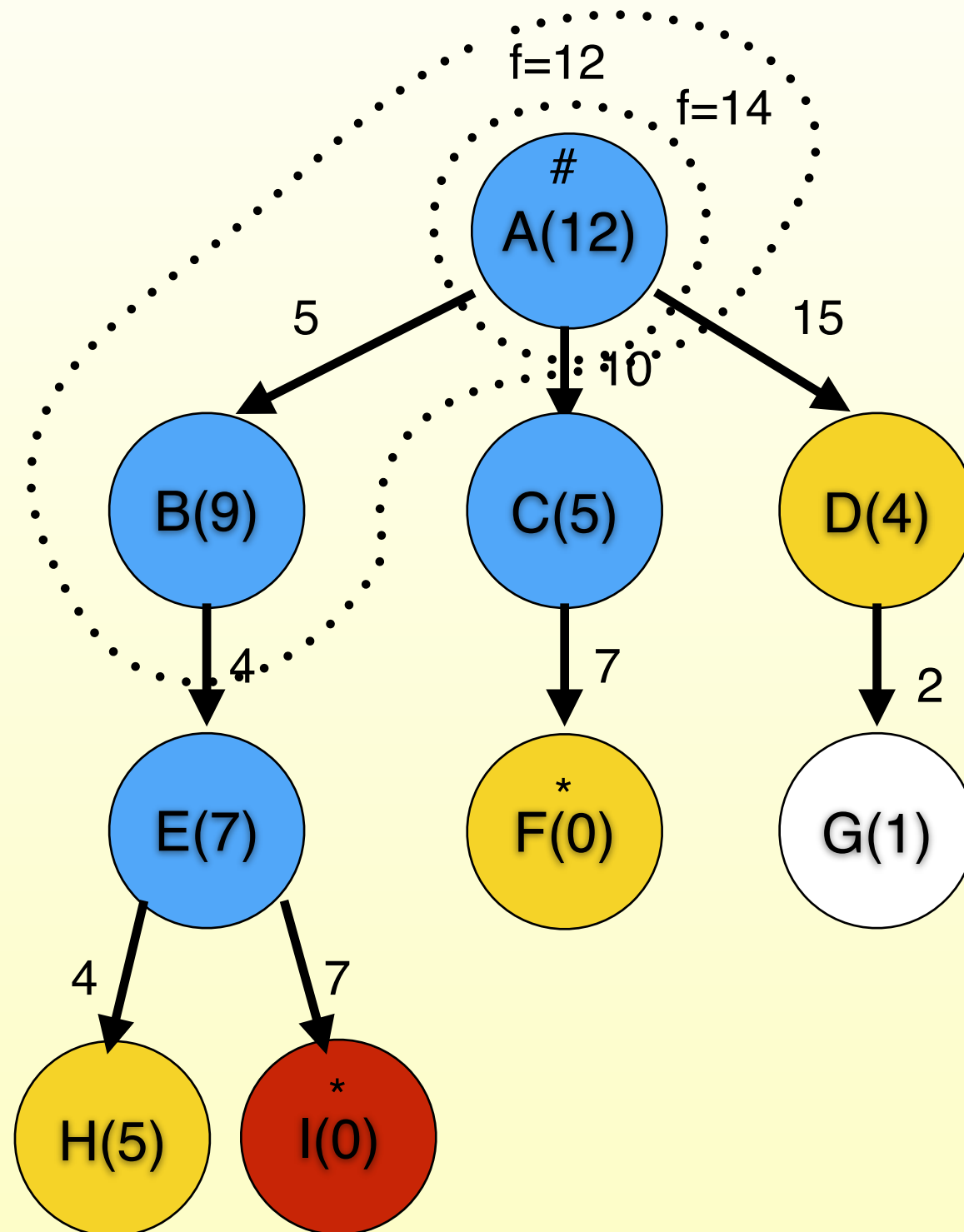
Contours

- ❖ **With a consistent heuristic, value of f never decreases along any path starting from the initial node**
 - ❖ also known as *monotonicity* of the function
 - ❖ almost all admissible heuristics show monotonicity
 - ❖ those that don't can be modified through minor changes
- ❖ **this property can be used to draw contours**
 - ❖ regions where the f -cost is below a certain threshold
 - ❖ with uniform cost search ($h = 0$), the contours are circular
 - ❖ the better the heuristics h , the narrower the contour around the optimal path

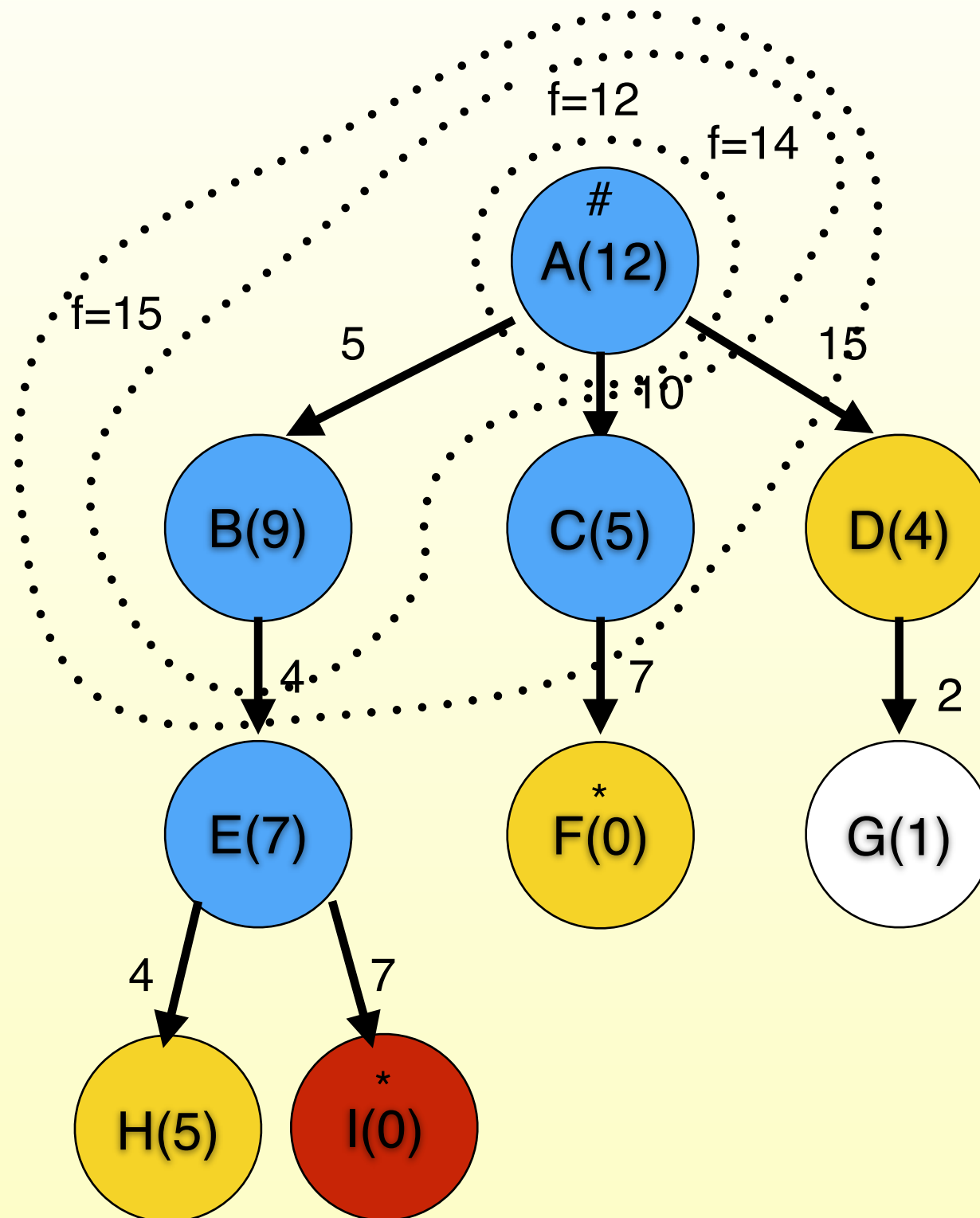
Visualizing contours



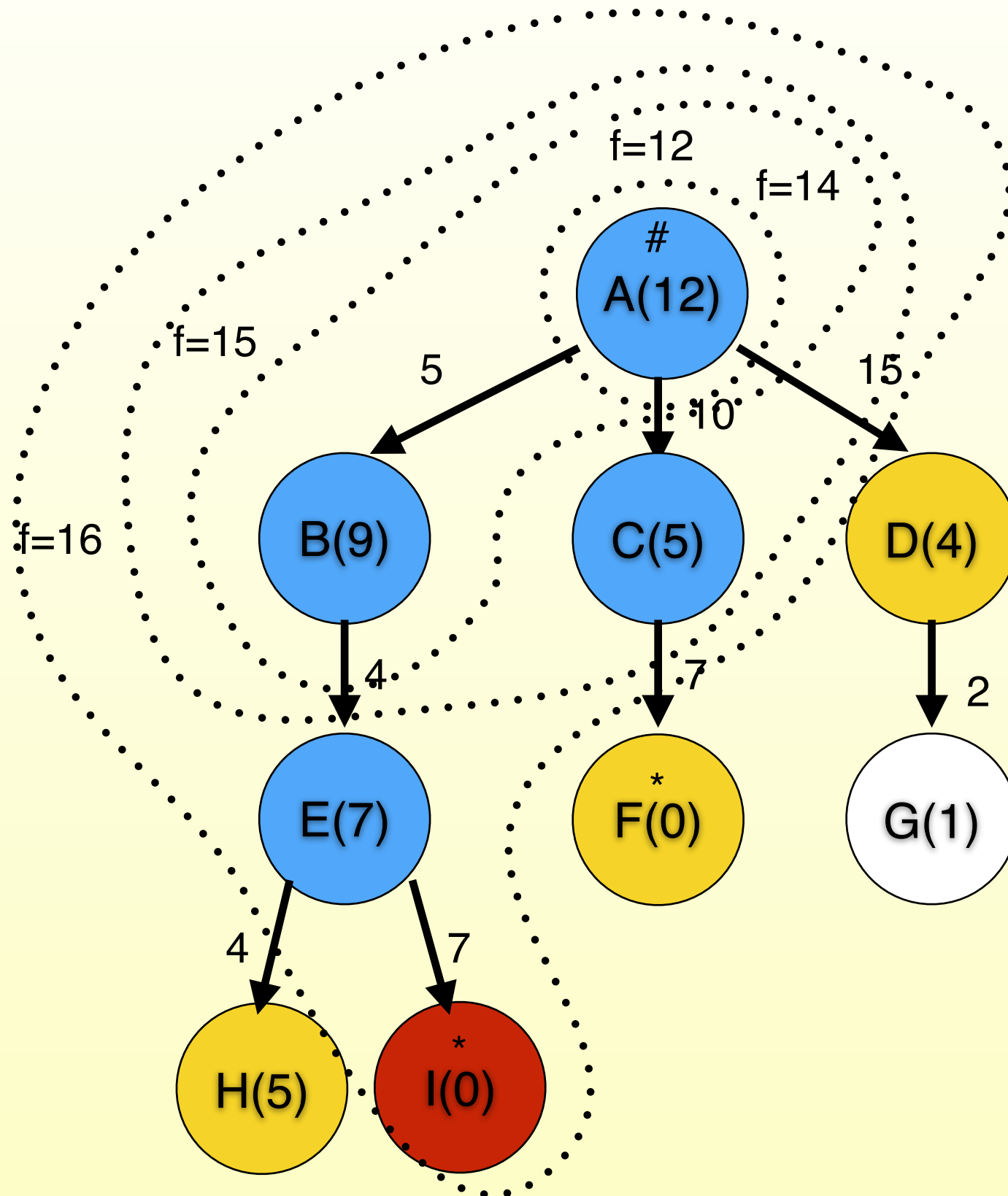
Visualizing contours



Visualizing contours



Visualizing contours



Complexity of A*

- ❖ **the number of nodes within the goal contour search space is still exponential**
 - ❖ with respect to the length of the solution
 - ❖ better than other algorithms, but still problematic
- ❖ **frequently, space complexity is more severe than time complexity**
 - ❖ A* keeps all generated nodes in memory
 - ❖ Memory-bounded variations (e.g. Iterative deepening A*) can alleviate the problem

Heuristics for Searching

- ❖ **for many tasks, a good heuristic is the key to finding a solution**
 - ❖ prune the search space
 - ❖ move towards the goal
- ❖ **In general, a heuristic with higher (more accurate) values yields better search efficiency than a heuristic with lower values**
 - ❖ As long as they are both admissible / consistent
 - ❖ If for all n , $h_1(n) \leq h_2(n)$, we say h_2 *dominates* h_1
 - ❖ This means A^* using h_2 expands $\leq$ nodes than A^* using h_1

8-Puzzle Heuristics

❖ level of difficulty

- ❖ around 20 steps for a typical solution
- ❖ branching factor is about 3
 - ❖ exhaustive search would be $3^{20} = 3.5 * 10^9$
- ❖ $9!/2 = 181,440$ different reachable states
 - ❖ distinct arrangements of 9 squares

❖ candidates for heuristic functions

- ❖ number of tiles in the wrong position
- ❖ sum of distances of the tiles from their goal position
 - ❖ city block or Manhattan distance

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

Admissible heuristics

❖ E.g., for the 8-puzzle:

- ❖ $h1(n)$ = number of misplaced tiles
- ❖ $h2(n)$ = total Manhattan distance
- ❖ (i.e., no. of squares from desired location of each tile)

- ❖ $h1(S) = 8$
 - ❖ there are only 8 tiles, the empty space doesn't count as tile
- ❖ $h2(S) = 3+1+2+2+2+3+3+2 = 18$
- ❖ Note: $h2$ dominates $h1$

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

How to generate heuristics?

❖ From relaxed versions of the problem

- ❖ This means we ignore some restrictions of the problem
- ❖ Equivalent to adding edges between previously non-neighboring states in the state space
- ❖ Examples: h_1 , h_2 from the previous slide

❖ From subproblems

- ❖ This means we consider some non-goal states as if they were goal states
- ❖ Example: have 1,2,3,4 in the right places in 8-Puzzle, no matter the location of 5,6,7,8
- ❖ Exact cost to subproblems can be calculated exactly and stored in a **pattern database**

❖ By combining other heuristics

- ❖ If h and h' are admissible, then $\max(h, h')$ is also admissible
- ❖ In some cases (but not always), we can also add heuristics (see **disjoint pattern databases**)

❖ From experience

- ❖ Human expertise
- ❖ Machine learning